

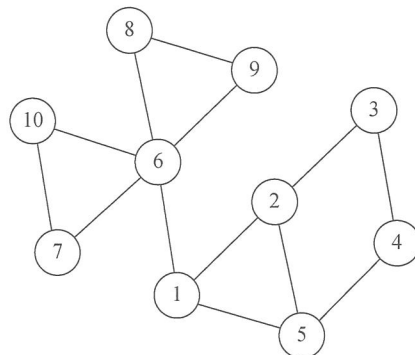
提高组 CSP-S 2024 初赛模拟卷 10

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 在 Linux 系统终端中，杀死名为 test 的进程的命令是（ ）。
A. ps test B. kill test C. stop test D. killall test
2. 机房的 IP 地址为 192.168.10.1/24，若想将其划分为 4 个子网，需要借的主机位数和每个子网的有效 IP 个数分别为（ ）。
A. 1; 126 B. 1; 127 C. 2; 64 D. 2; 62
3. 下列语句中，可以用于清空 `stack<int> stk` 的是（ ）。
A. `stk.clear();`
B. `while(stk.empty()) stk.pop();`
C. `while(stk.size()) stk.pop();`
D. `clear(stk);`
4. 从 0~9 中选择 3 个数构成不含前导 0 且能被 3 整除的数的方案数为（ ）。
A. 192 B. 256 C. 48 D. 228
5. 在一张无权图中使用广度优先搜索求从 1 号结点到其他所有结点的最短路径，下列说法中错误的是（ ）。
A. 某一时刻，队列中某两个结点到结点 1 的距离的差为 0
B. 某一时刻，队列中某两个结点到结点 1 的距离的差为 1
C. 某一时刻，队列中某两个结点到结点 1 的距离的差为 2
D. 某一时刻，队列中任意两个结点到结点 1 的距离差为 0
6. 关于图算法 BFS、Dijkstra 和 Bellman-Ford，求解边权均为 -1 的图的最长路径可以使用（ ）种算法，求解边权均为 1 的图的最长路径可以使用（ ）种算法，求解边权均小于 0 的图的最长路径可以使用（ ）种算法。（上述 3 个问题中若有不存在解的情况，所选算法需要判断不存在解的情况，只考虑正确性而不考虑效率。）
A. 3; 1; 3 B. 1; 3; 1 C. 1; 2; 3 D. 3; 2; 1

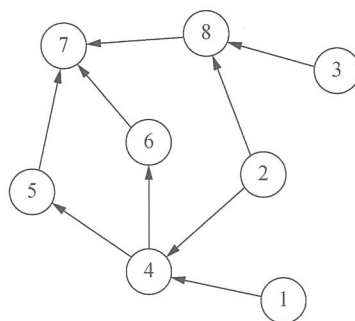
7. 在序列分块类问题中, 对长度为 n 的序列, 若取块长 B , 零散块的修改复杂度为 $O(B)$, 查询复杂度为 $O(\log^2 n)$, 整块的修改复杂度为 $O(1)$, 查询复杂度为 $O\left(\frac{n}{B} \log^2 n\right)$, 则 m 次修改/查询的最坏复杂度最优为 ()。
- A. $O(m\sqrt{n} \log^2 n)$ B. $O(m\sqrt{n} \log n)$
 C. $O(m\sqrt{n \log n})$ D. $O(m\sqrt{n})$
8. 在 C++ 程序中执行如下代码片段后, t 的值为 ()。
- ```
int a = 1, b = 1, c = 4, d = 5, e = 1;
bool f = true;
auto t = ((a > b ^ c) && f) < d ^ e;
```
- A. true                      B. false                      C. 0                      D. 1
9. 在一口无限深的井中, 蜗牛处于井口下方 5 个单位长度处, 每天白天以  $\frac{1}{2}$  的概率向上爬 3 个单位长度, 晚上下滑 1 个单位长度, 则蜗牛爬出井口 (向上爬大于等于 5 个单位即为爬出井口) 的期望天数为 ( )。
- A.  $\frac{8}{3}$                       B.  $\frac{12}{5}$                       C. 10                      D. 5
10. 使用二进制分组优化多重背包问题, 数量为 10 的物品会被拆分为 ( ) 件物品。
- A. 1                      B. 2                      C. 3                      D. 4
11. 有一个数列定义如下:  $a_0 = 1, a_1 = 1, a_n = (a_{\{n-1\}} + a_{\{n-2\}}) \bmod 10$  ( $n > 1$ ), 则  $a_{1024}$  和  $a_{2048}$  分别等于 ( )。
- A. 3; 1                      B. 5; 4                      C. 8; 5                      D. 1; 1
12. 下列代码片段中, 有 ( ) 种方法可以判断状态  $s$  的第  $x$  个二进制位的情况 (0/1) ( $x$  从 0 开始编号, 编号从低位到高位增大)。
- (1)  $(\text{bool})(s \gg x \& 1)$   
 (2)  $(\text{bool})(s \ll x \& 1)$   
 (3)  $(\sim s \gg x | \sim 1) == s$   
 (4)  $(\sim s | \sim 1 \ll x) == s$
- A. 1                      B. 2                      C. 3                      D. 4

13. 下图中点双连通分量和边双连通分量分别有 ( ) 个。



- A. 1; 1      B. 2; 3      C. 3; 2      D. 2; 2

14. 下图共有 ( ) 种不同的拓扑序。



- A. 35      B. 21      C. 24      D. 76

15. 由 3 个中括号和 3 个小括号组成的本质不同的合法括号序列数为 ( ), 其中, 一对匹配的小括号之间不应有中括号。

- A. 770      B. 710      C. 192      D. 560

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填√, 错误填×; 除特殊说明外, 判断题每题 1.5 分, 选择题每题 3 分, 共计 40 分)

(1)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 const int N = 2e5 + 10;
04
05 long long a[N];
06
```

```
07 int main() {
08 int n, m;
09 scanf("%d %d", &n, &m); // 1 <= n, m <= 2e5
10 for(int i = 1; i <= n; ++ i) scanf("%d", a + i);
 // 1 <= a[i] <= 1e9
11 while(m --) {
12 int x;
13 scanf("%d", &x); // 1 <= x <= 1e9
14 int lst = 1;
15 for(int i = 1; i <= n; ++ i) {
16 if(a[i] >= lst) {
17 if(x <= a[i]) {
18 a[i] += x - lst + 1;
19 break;
20 } else {
21 int dis = a[i] - lst + 1;
22 lst = a[i] + 1;
23 a[i] += dis;
24 }
25 }
26 }
27 }
28 }
29
30 for(int i = 1; i <= n; ++ i) printf("%lld\n", a[i]);
31 return 0;
32 }
```

假定输入数据范围:  $1 \leq n, m \leq 2 \times 10^5$ ,  $a_i, r \in [1, 10^9]$ 。

#### ■ 判断题

16. 将第 5 行中的 long long 改为 int, 不影响程序结果。 ( )
17. (2 分) 该算法的时间复杂度为  $O(nm)$ 。 ( )
18. 当输入为 3 2 3 2 5 6 1 时, 输出为 7 2 7。 ( )
19. 当输入为 3 2 3 2 1 6 6 时, 输出为 9 2 1。 ( )

#### ■ 选择题

20. 当输入为 4 5 10 20 30 40 20 10 30 25 5 时, 输出为 ( )。
- A. 10 20 30 40                      B. 100 20 30 40
- C. 30 30 60 100                    D. 90 30 30 40

21. 当输入为 3 4 10 20 10 10 20 10 10 时, 输出为 ( )。
- A. 10 20 60      B. 10 70 10      C. 60 20 10      D. 10 20 10

(2)

```
01 #include <bits/stdc++.h>
02
03 using namespace std;
04
05 int cow[300010];
06 int buf[300010], tot;
07 int buf2[300010], tot2;
08
09 signed main() {
10 int n;
11 scanf("%d", &n);
12 for(int i = 1; i <= n; ++ i) {
13 char ch;
14 for(ch = getchar(); ch != '0' && ch != '1'; ch = getchar());
15 cow[i] = ch - '0';
16 }
17 int tim = n << 1, start = 0;
18 for(int i = 1; i <= n + 1; ++ i) {
19 if(cow[i] == 1 && cow[i - 1] == 0) start = i;
20 else if(cow[i] == 0 && cow[i - 1] == 1) {
21 if(start == 1 || i == n + 1) {
22 buf2[++ tot2] = i - start;
23 tim = min(tim, (i - start - 1) << 1 | 1);
24 } else {
25 buf[++ tot] = i - start;
26 tim = min(tim, ((i - start - 1) >> 1) << 1 | 1);
27 }
28 }
29 }
30 long long ans = 0;
31 for(int i = 1; i <= tot; ++ i) ans += (buf[i] + tim - 1) / tim;
32 for(int i = 1; i <= tot2; ++ i) ans += (buf2[i] + tim - 1) / tim;
33 cout << ans << endl;
34 return 0;
35 }
```

输入为一个整数  $n$  以及一个长度为  $n$  的 01 字符串。保证  $1 \leq n \leq 3 \times 10^5$ 。



## ■ 判断题

22. 在第 18~29 行的循环结束后, 始终有  $\text{tot} + \text{tot2} == n$  为 true。 ( )
23. 若输入为 5 11111, 则输出为 1。 ( )
24. 若输入为 6 011101, 则输出为 4。 ( )

## ■ 选择题

25. 若输入为 9 101001101, 则  $\text{tot2} = ( )$ 。  
A. 0                      B. 1                      C. 2                      D. 3
26. 若输入为 15 110011100101101, 则  $\text{buf}[1 \dots \text{tot}]$  的内容为 ( )。  
A. 3 1 1                  B. 3 1 2                  C. 1 1 2 1                  D. 1 1 2 2
27. 若输入为 13 0010110111011, 则  $\text{tot} + \text{tot2} = ( )$ 。  
A. 1                      B. 2                      C. 3                      D. 4

(3)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 const int N = 5e5 + 10;
04 int pls[N], cnt[N], dist[N], dist2[N];
05 signed main() {
06 ios::sync_with_stdio(false);
07 cin.tie(NULL);
08 int n, k;
09 cin >> n >> k; // 1 <= k <= n <= 5e5
10 int x;
11 for(int i = 1; i <= k; ++ i) {
12 cin >> x;
13 pls[x] = i;
14 ++ cnt[x];
15 }
16 int ans = 0;
17 for(int i = 1; i <= k; ++ i) {
18 cin >> x;
19 ++ cnt[x];
20 if(pls[x]) {
21 int d;
```

```
22 if(pls[x] < i) {
23 d = k + pls[x] - i;
24 }
25 else d = pls[x] - i;
26 ans = max(ans, ++ dist[d]);
27 if(pls[x] < k + 1 - i) d = k + pls[x] - (k + 1 - i);
28 else d = pls[x] - (k + 1 - i);
29 ans = max(ans, ++ dist2[d]);
30 }
31 }
32 for(int i = 1; i <= n; ++ i)
33 if(!cnt[i])
34 ++ ans;
35 cout << ans << endl;
36 return 0;
37 }
```

假定输入数据中  $1 \leq k \leq n \leq 5 \times 10^5$ ，后跟的两个长度为  $k$  的序列分别是  $\{1, 2, \dots, n\}$  的一个大小为  $k$  的子集的一个排列。

■ 判断题

28. 若输入为 6 3 1 2 3 2 3 1，则输出为 6。 ( )
29. 若输入为 6 3 1 2 3 4 5 6，则输出为 1。 ( )
30. (2 分) 若输入为 6 4 1 2 3 4 4 3 2 5，则执行到第 32 行时  $\text{cnt}[1 \dots n]$  的值为 1 2 2 2 1 0。 ( )

■ 选择题

31. 若输入为 6 4 1 2 3 4 4 3 1 5，则第 23 行执行 ( ) 次。  
A. 1                      B. 2                      C. 3                      D. 4
32. 若输入为 5 4 1 2 3 4 4 3 1 5，则输出为 ( )。  
A. 1                      B. 2                      C. 3                      D. 4
33. 若输入为 5 3 1 2 3 2 3 4，则执行完第 17~31 行的 for 循环后  $\text{dist2}[2] =$  ( )。  
A. 0                      B. 1                      C. 2                      D. 3

### 三、完善程序（单选题，每小题 3 分，共计 30 分）

(1) (击靶) 在坐标轴上  $[-C, C]$  区间内有  $T$  个靶子（不重合），射手刚开始处在坐标原点，进行  $C$  次操作。

L: 从坐标  $x$  处移动到坐标  $x-1$  处。

R: 从坐标  $x$  处移动到坐标  $x+1$  处。

F: 进行一次射击，若当前坐标处存在未被击倒的靶子，则击倒该靶子。

求对原操作序列修改一次后，最多能击倒多少靶子？

输入格式：

第 1 行是两个正整数  $T$  和  $C$ 。第 2 行是  $T$  个正整数，表示靶子的坐标。第 3 行是一个长度为  $C$  的字符串，字典为  $\{L, R, F\}$ ，表示原操作序列。

解题思路：

从后往前枚举改变的具体时间和改变的操作类型。具体地说，对于同一种操作改变（例如，L 操作改成 R 操作），如果时间  $x$  之前的某个时间  $y$  发生了改变，则不管  $y$  多大，时间  $x$  所处的位置都是固定的一个值。可以预处理出这个位置，辅助变化位置的枚举。

```
01 #include <bits/stdc++.h>
02
03 using namespace std;
04
05 const int N = 1e5 + 10, DELTA = 1e5 + 1;
06 int pls[N], mp[N << 1], lft[N << 1], second[N << 1], LEFT[N << 1],
 nTar, nOp, curAns, ans;
07 char op[N];
08
09 void work(int delta, char from, char to) {
10 memset(second, 0, sizeof(second));
11 memcpy(lft, LEFT, sizeof(lft));
12 for(①) {
13 if(②) {
14 if(from == 'F' && mp[pls[i] + DELTA] && lft[pls[i] + DELTA]
 == i && second[pls[i] + DELTA] == 0)
15 ans = max(ans, curAns - 1);
16 else if(to == 'F' && mp[pls[i] + DELTA] && lft[pls[i] +
 DELTA] == 0)
17 ans = max(ans, curAns + 1);
```



```
18 else
19 ans = max(ans, curAns);
20 }
21 if(op[i] == 'F') {
22 if(mp[pls[i] + DELTA] && lft[pls[i] + DELTA] == i) {
23 lft[pls[i] + DELTA] = second[pls[i] + DELTA];
24 if(lft[pls[i] + DELTA] == 0)
25 -- curAns;
26 }
27 pls[i] += delta;
28 if(mp[pls[i] + DELTA]) {
29 if(lft[pls[i] + DELTA] == 0) {
30 ③;
31 lft[pls[i] + DELTA] = i;
32 }
33 ④;
34 lft[pls[i] + DELTA] = min(lft[pls[i] + DELTA], i);
35 }
36 pls[i] -= delta;
37 }
38
39 }
40 }
41
42 signed main() {
43 ios::sync_with_stdio(false);
44 cin.tie(NULL);
45 cin >> nTar >> nOp; // nTar 表示靶子的个数, nOp 表示操作序列的长度
46 int tar;
47 for(int i = 1; i <= nTar; ++ i) {
48 cin >> tar; // 目标的位置
49 mp[tar + DELTA] = 1;
50 }
51 cin >> (op + 1); // 输入操作序列
52 int x = 0;
53 for(int i = 1; i <= nOp; ++ i) {
54 pls[i] = x;
55 if(op[i] == 'L') -- x;
56 else if(op[i] == 'R') ++ x;
57 else if(mp[x + DELTA] && !lft[x + DELTA]) {
58 ++ ans;
```

```
59 lft[x + DELTA] = i;
60 }
61 }
62 int tmp = ans;
63 memcpy(LEFT, lft, sizeof(lft));
64 curAns = tmp; work(2, 'L', 'R');
65 curAns = tmp; work(-2, 'R', 'L');
66 curAns = tmp; work(1, 'L', 'F');
67 curAns = tmp; work(⑤);
68 curAns = tmp; work(-1, 'F', 'L');
69 curAns = tmp; work(1, 'F', 'R');
70 cout << ans << endl;
71 return 0;
72 }
```

34. ①处应填 ( )。

- A. `int i = 1; i <= nOp; ++ i`    B. `int i = nOp; i >= 1; --i`  
C. `int i = 1; i <= nTar; ++ i`    D. `int i = nTar; i >= 1; -- i`

35. ②处应填 ( )。

- A. `op[i] == to`    B. `op[i] == from`  
C. `op[i] == 'L'`    D. `op[i] == 'R'`

36. ③处应填 ( )。

- A. `++ curAns`    B. `-- curAns`  
C. `++ ans`    D. `--ans`

37. ④处应填 ( )。

- A. `LEFT[pls[i] + DELTA] = i`  
B. `ans = curAns`  
C. `second[pls[i] + DELTA] = i`  
D. `++ curAns`

38. ⑤处应填 ( )。

- A. `1, 'R', 'F'`    B. `2, 'R', 'F'`  
C. `-1, 'R', 'L'`    D. `-1, 'R', 'F'`

- (2) (字典序最小最长路径) 给出一张有  $n$  个结点、 $m$  条边的带权无环有向图, 边以  $(a_i, b_i, l_i)$  的形式给出, 表示从  $a_i$  到  $b_i$  的一条有向边, 边权为  $l_i$ 。现在要求对于每个结点, 输出以它为起点且经过最多条边的路径经过的边数以及边权之和。如果有多条经过边数最多的路径, 则选择将路径边权按顺序排列, 得到序列中字典序最小的一条。

数据范围:

$$1 \leq n \leq 2 \times 10^5, 1 \leq m \leq 4 \times 10^5, 1 \leq a_i, b_i \leq 2 \times n, 1 \leq l_i \leq 10^9。$$

输出格式:

第 1 行是两个正整数  $n, m$ 。第 2 行到第  $n+1$  行每行 3 个整数  $a_i, b_i, l_i$ , 代表一条边。

解题思路:

输入为一个 DAG, 使用记忆化搜索来确定以某个结点为起点的最长且边权字典序最小的路径。在确定后继结点的选择时, 首先考虑路径长度最大, 而当多条路径的后继结点路径长度一样大时, 考虑将相同长度的结点按照(最小后继边权, 最优后继结点的哈希值)双关键字排序得到当前长度的结点的顺序, 作为当前结点的哈希值(也即结点在相同最大路径长度的结点中的排序)。这样在做后继结点的决策时, 只需要选择同深度结点中哈希值最小的结点即可。

```
01 #include <bits/stdc++.h>
02
03 using namespace std;
04
05 const int N = 2e5 + 10;
06 vector<pair<int,int>> g[N];
07 int dp[N], nexE[N], nexT[N], rnk[N], id[N];
08 long long sum[N];
09 bool vis[N];
10
11 void work(int cur) {
12 if(vis[cur]) return ;
13 vis[cur] = true;
14 for(auto it : g[cur]) {
15 work(it.first);
16 dp[cur] = max(dp[cur], dp[it.first] + 1);
17 }
18 for(auto it : g[cur])
19 if(dp[it.first] + 1 == dp[cur] && (nexE[cur] == 0 || it.second
20 < nexE[cur]))
21 ①;
```

```
21 }
22
23 signed main() {
24 ios::sync_with_stdio(false);
25 cin.tie(NULL);
26 int n, m;
27 cin >> n >> m; // 输入结点数 n 和边数 m
28 int u, v, w;
29 for(int i = 1; i <= n; ++ i) {
30 cin >> u >> v >> w; // 从 u 到 v 的一条有向边, 边权为 w
31 g[u].push_back(make_pair(v, w));
32 }
33 for(int i = 1; i <= n; ++ i)
34 if(②)
35 work(i);
36 for(int i = 1; i <= n; ++ i)
37 id[i] = i;
38 sort(id + 1, id + n + 1, [&](int x, int y) { return ③ ; });
39 vector<int> vec;
40 for(int i = 1; i <= n; ++ i) {
41 if(dp[id[i]] == 0) continue;
42 if(dp[id[i]] != dp[id[i - 1]] && vec.size()) {
43 sort(vec.begin(), vec.end(), [&](int x, int y) {
44 return nexE[x] < nexE[y] || (④ && rnk[nexT[x]]
45 < rnk[nexT[y]]);
46 });
47 int cnt = 0;
48 for(int x : vec) rnk[x] = ++ cnt;
49 vec.clear();
50 }
51 vec.push_back(id[i]);
52 for(auto it : g[id[i]])
53 if(dp[it.first] + 1 == dp[id[i]] && nexE[id[i]] == it.second
54 && (nexT[id[i]] == 0 || rnk[nexT[id[i]]] > rnk[it.fi]))
55 nexT[id[i]] = it.first;
56 sum[id[i]] += ⑤;
57 }
58 for(int i = 1; i <= n; ++ i)
59 cout << dp[i] << ' ' << sum[i] << endl;
60 return 0;
61 }
```

39. ①处应填 ( )。
- A. `nexE[cur] = it.second`                      B. `nextE[cur] = dp[it.first] + 1`  
C. `nexT[cur] = it.second`                      D. `nexT[cur] = it.first`
40. ②处应填 ( )。
- A. `dp[i]`                      B. `!vis[i]`                      C. `!rnk[i]`                      D. `nexT[i]`
41. ③处应填 ( )。
- A. `nexE[x] < nexE[y]`                      B. `nexT[x] < nexT[y]`  
C. `dp[x] < dp[y]`                      D. `id[x] < id[y]`
42. ④处应填 ( )。
- A. `rnk[x] == rnk[y]`                      B. `rnk[x] < rnk[y]`  
C. `nexE[x] > nexE[y]`                      D. `nexE[x] == nexE[y]`
43. ⑤处应填 ( )。
- A. `sum[id[i]] + nexE[id[i]]`  
B. `sum[nexT[id[i]]] + nexE[id[i]]`  
C. `sum[rnk[id[i]] + nexE[id[i]]`  
D. `nexE[id[i]] + nexT[id[i]]`